



Debre Berhan University
College of Computing
Department
of
Information Technology

Course Title: Advanced Programming

Course Code: ITec 3057

Credit : 3 Cr. Hrs / 5 ECTS (2 Hrs. Lec. & 3 Hrs. Lab)

Target group: 3rd year Information Technology

Course Manager: Yirga G.

Chapter 3:

Network Programming

Contents

- Network programming basics
 - Client-Server Architecture
 - Identifying a machine
 - Servers and Clients
 - Ports and sockets
 - TCP and UDP
 - Socket programming
 - JAVA.NET CLASS
 - Multithreaded Server
 - URL
 - Network Interface

Introduction

- Network
 - A network is a group or system of interconnected elements that allow for the sharing of information, resources, or connections.
- Network programming
 - Network programming is the practice of writing computer code to create programs that can communicate and exchange data with other programs over a network.
 - It involves using various programming languages, libraries, and protocols to enable this communication.

Introduction

- Client server architecture
 - A **client is a device or program** that **requests services** or resources from a server, while the **server is a computer or system** that **provides those services**.
 - In client-server architecture, a client device (like a computer, smartphone, or web browser) requests information or services from a centralized server, which then processes the request and sends back the requested data or service.
 - This architecture is a fundamental model for many network applications, enabling efficient resource sharing and management.

Introduction

- Host and Port Number
 - In networking, a **host is a device** that **runs applications and services**, while a **port number is a 16-bit number** that **identifies a specific application or service** on that host.
 - A host, such as a server, often runs multiple processes or applications such as web and email.
 - Each host maintains a table that maps the port numbers to the processes; when a data packet arrives at a host, the host uses this table to route the packet to the correct process or service.

Introduction

- IP Address and DNS
 - An IP address is a unique numerical identifier assigned to each device connected to a network, like the internet, that allows it to communicate with other devices.
 - DNS (Domain Name System) is a hierarchical and decentralized system that translates human-readable domain names (like google.com) into numerical IP addresses that computers use to communicate.

Introduction

- **Socket**

- In networking, a **socket is an endpoint for communication between two programs**, typically on different computers, over a network.
- It's a combination of an IP address and a port number, identifying a specific application process.
- Sockets enable two-way communication, allowing data to be exchanged between the connected programs.

Introduction

- **Types of Socket**
 - **Stream sockets** : provide reliable, sequenced, and unduplicated data transfer using TCP
 - **Datagram socket**: offer bidirectional communication using UDP, but with no guaranteed order or reliability.
 - **Raw sockets**: allow direct access to IP packets, enabling applications to work with lower-level networking protocols.

SOCKET PROGRAMMING

- A socket is an endpoint of a two-way communication link between two programs running on the network.
- Socket is bound to a port number so that the TCP layer can identify the application that data is destined to be sent.
- Java provides a set of classes, defined in a package called `java.net`, to enable the rapid development of network applications.
- Key classes, interfaces, and exceptions in `java.net` package simplifying the complexity involved in creating client and server programs

Java.net *Classes*

- ContentHandler
- DatagramPacket
- DatagramSocket
- DatagramSocketImpl
- HttpURLConnection
- InetAddress
- MulticastSocket
- ServerSocket
- Socket
- SocketImpl
- URL
- URLConnection
- URLEncoder
- URLStreamHandler

The Interfaces

- ContentHandlerFactory
- FileNameMap
- SocketImplFactory
- URLStreamHandlerFactory

Exceptions

- BindException
- ConnectException
- MalformedURLException
- NoRouteToHostException
- ProtocolException
- SocketException
- UnknownHostException
- UnknownServiceException

Internet addresses

- **Static construction using a factory method**
 - **InetAddress getByName(String hostName)**
 - hostName can be “www.dbu.edu.et”, or
 - hostName can be “213.55.96.150”
 - **InetAddress getLocalHost()**
- **Some useful methods:**
 - **String getHostName()**
 - Gives you the host name (for example “www.dbu.edu.et”)
 - **String getAddress()**
 - Gives you the address (for example “213.55.96.150”)
 - **InetAddress getLocalHost()**
 - **InetAddress[] getAllByName(String hostName)**

Internet addresses ...

- `import java.net.*;`
- `public static void main(String[] args)`
- `{`
 - `try {`
 - `InetAddress inet1 = InetAddress.getByName("www.dbu.edu.et");`
 - `System.out.println("HostAddress=" + inet1.getHostAddress());`
 - `InetAddress inet2 = InetAddress.getByName("213.55.96.150");`
 - `System.out.println("HostName=" + inet2.getHostName());`
 - `if (inet1.equals(inet2))`
 - `System.out.println("Addresses are equal");`
 - `}`
 - `catch (UnknownHostException uhe) {`
 - `uhe.printStackTrace();`
 - `}`
- `}`
- `}`

TCP Sockets

- Connection-oriented
- The client and server programs both bind a **socket** to each end of the connection and communicate by reading and writing to those sockets.
- Java supports client/server computing using sockets.
 - **Socket** for clients and
 - **ServerSocket** for servers in a connection-oriented environment.

Server Sockets

- Creating and using server sockets.
- **Constructors**
 - `ServerSocket(int port)`
 - `ServerSocket(int port, int backlog)`
- **Methods Description**
 - `accept()` :- Waits for a connection request and returns a `Socket`
 - `close()` :- Stops waiting for requests from clients

Client Sockets

- Creating client sockets
- **Constructor**
 - public Socket()
 - Socket(String host, int port)
- **Methods Description**
 - `getInputStream()`:- Returns an `InputStream` for receiving data
 - `getOutputStream()`:- Returns an `OutputStream` to send data
 - `close()`:- Closes the socket connection

Life cycles of the client/server model

- **Life cycle of a client:**
 - A client socket is created using `Socket()`
 - The socket connects to a remote host
 - Data exchange between local and remote hosts.
 - an input stream to read data
 - an output stream to send data to server.
 - Connection terminates after transmission of data is complete.

Life cycle of a server:

- A server socket is created on a particular port using `ServerSocket()`.
- The socket listens for an incoming connection request from a client on the port using `accept()`.
- Data exchange after connection is made.
 - The server and client interact according to an agreed upon protocol.
- Either or both of `getInputStream()` and `getOutputStream()`, depending on the type of server, is called to exchange data with the client.
- The server, the client, or both closes the connection upon completion of data exchange.
- The server returns to step 2 and waits for the next connection request.

Example: - Java server (TCP)

```
package tcpserver;
import java.io.*;
import java.net.*;
public class TCPServer {
    public static void main(String[] args) throws Exception{
        int port = 1234; // Server will listen on this port
        try (ServerSocket serverSocket = new ServerSocket(port)) {
            System.out.println("Server is listening on port " + port);
            while (true) {
                Socket socket = serverSocket.accept();
                System.out.println("New client connected");

                InputStream input = socket.getInputStream();
                BufferedReader reader = new BufferedReader(new InputStreamReader(input));
```

Example: - Java server (TCP)...

```
OutputStream output = socket.getOutputStream();
PrintWriter writer = new PrintWriter(output, true);
String text;
while ((text = reader.readLine()) != null) {
    System.out.println("Received: " + text);
    if (text.equalsIgnoreCase("bye")) {
        writer.println("Goodbye!");
        break;
    }
    writer.println("Server: " + text.toUpperCase()); // Echo back in uppercase
}
socket.close();
System.out.println("Connection closed");
}
} catch (IOException ex) {
    System.out.println("Server exception: " + ex.getMessage());
    ex.printStackTrace();
}
}
```

Example: Java client (TCP)

```
package tcpclient;
import java.io.*;
import java.net.*;
import java.util.Scanner;
public class TCPClient {

    public static void main(String[] args) throws Exception {
        String hostname = "localhost";
        int port = 1234;

        try (Socket socket = new Socket(hostname, port)) {
            OutputStream output = socket.getOutputStream();
            PrintWriter writer = new PrintWriter(output, true);

            InputStream input = socket.getInputStream();
            BufferedReader reader = new BufferedReader(new InputStreamReader(input));

            Scanner scanner = new Scanner(System.in);
            String text;

            System.out.println("Connected to the server. Type messages (type 'bye' to quit):");
```


Example: Java client (TCP) ...

```
do {  
    System.out.print("Client: ");  
    text = scanner.nextLine();  
  
    writer.println(text);  
  
    String response = reader.readLine();  
    System.out.println(response);  
  
} while (!text.equalsIgnoreCase("bye"));  
  
socket.close();  
} catch (UnknownHostException ex) {  
    System.out.println("Server not found: " + ex.getMessage());  
} catch (IOException ex) {  
    System.out.println("I/O error: " + ex.getMessage());  
}  
}  
}
```

Multithreaded Server

- The multithreaded server program creates a new thread for every client request.
- This way each client has its own connection to the server for passing data back and forth.
- **Basic Flow**
 - while (true) {
 - accept a connection ;
 - create a thread to deal with the client ;
 - end while
 - }

Multithreaded Server side code:

- package multithreadedtcpserver;
- import java.io.*;
- import java.net.*;
- public class MultiThreadedTCPServer {
- public static void main(String[] args) {
- final int PORT = 9999;
-
- try (ServerSocket serverSocket = new ServerSocket(PORT)) {
- System.out.println("Server started on port " + PORT);
-
- while (true) {
- Socket clientSocket = serverSocket.accept();
- System.out.println("New client connected: " + clientSocket);
-
- // Handle client in a new thread
- new Thread(new ClientHandler(clientSocket)).start();
- }
- } catch (IOException e) {
- e.printStackTrace();
- }
- }
- }

Multithreaded Server

- package multithreadedtcpserver;
- import java.io.*;
- import java.net.*;
- public class ClientHandler implements Runnable {
- private Socket socket;
- ClientHandler(Socket socket) {
- this.socket = socket;
- }
- public void run() {
- try (
- BufferedReader reader = new BufferedReader(new InputStreamReader(socket.getInputStream()));
- PrintWriter writer = new PrintWriter(socket.getOutputStream(), true)
-) {
- String message;
- while ((message = reader.readLine()) != null) {
- System.out.println("Received: " + message);
- writer.println("Server received: " + message);
- }
- } catch (IOException e) {
- System.out.println("Client disconnected: " + socket);
- }
- }
- }

Client side code

- package tcpclientguimulti;
- import javax.swing.*;
- import java.awt.*;
- import java.awt.event.*;
- import java.io.*;
- import java.net.*;
- public class TCPClientGUIMulti extends JFrame {
- private JTextField inputField;
- private JTextArea chatArea;
- private JButton sendButton;
-
- private Socket socket;
- private PrintWriter writer;
- private BufferedReader reader;
-
- public TCPClientGUIMulti() {
- setTitle("TCP Client GUI");
- setSize(500, 350);
- setDefaultCloseOperation(EXIT_ON_CLOSE);
- setLayout(new BorderLayout());

Client side code

- `inputField = new JTextField();`
- `chatArea = new JTextArea();`
- `chatArea.setEditable(false);`
- `sendButton = new JButton("Send");`
- `add(inputField, BorderLayout.NORTH);`
- `add(new JScrollPane(chatArea), BorderLayout.CENTER);`
- `add(sendButton, BorderLayout.SOUTH);`
- `sendButton.addActionListener(e -> sendMessage());`
- `inputField.addActionListener(e -> sendMessage());`
- `connectToServer();`
- `setVisible(true);`
- `}`

Client side code

- `private void connectToServer() {`
- `try {`
- `socket = new Socket("localhost", 9999);`
- `writer = new PrintWriter(socket.getOutputStream(), true);`
- `reader = new BufferedReader(new InputStreamReader(socket.getInputStream()));`
-
- `// Start a background thread to listen for server messages`
- `new Thread(() -> {`
- `String response;`
- `try {`
- `while ((response = reader.readLine()) != null) {`
- `chatArea.append("Server: " + response + "\n");`
- `}`
- `} catch (IOException ex) {`
- `chatArea.append("Connection closed.\n");`
- `}`
- `}).start();`
-
- `} catch (IOException e) {`
- `chatArea.append("Failed to connect to server: " + e.getMessage() + "\n");`
- `}`
- `}`

Client side code

- `private void sendMessage() {`
- `String message = inputField.getText().trim();`
- `if (!message.isEmpty() && writer != null) {`
- `chatArea.append("Client: " + message + "\n");`
- `writer.println(message);`
- `inputField.setText("");`
- `}`
- `}`
-
- `public static void main(String[] args) {`
- `SwingUtilities.invokeLater(TCPClientGUIMulti::new);`
- `}`
-
- `}`

UDP Sockets

- Connectionless
- UDP is contained in two classes in java.net package:-
 - DatagramSocket
 - DatagramPacket
- **Datagram packet:** -
 - is an independent, self-contained message sent over the network whose
 - **arrival, arrival time, and content are not guaranteed.**

DatagramPacket

- The class DatagramPacket contains several constructors
 - `public DatagramPacket(byte[] data, int length)` - for receiving
 - `public DatagramPacket(byte[] data, int length, InetAddress iaddr, int iport)` - for sending
- The key methods of DatagramPacket class are:
 - `byte[] getData()`:-Returns the data buffer.
 - `int getLength()`:- Returns the length of the data to be sent or the length of the data received.
 - `void setData(byte[] buf)`:- Sets the data buffer for this packet.
 - `void setLength(int length)`:- Sets the length for this packet.

DatagramSocket

- constructors
 - public DatagramSocket() throws SocketException
 - public DatagramSocket(int port) throws SocketException
 - public DatagramSocket(int port, InetAddress laddr) throws SocketException
- The key methods are:
 - void send(DatagramPacket p):- Sends a datagram packet from this socket.
 - void receive(DatagramPacket p):- Receives a datagram packet from this socket.

UDP Server Code

- `// UDPServer.java: A simple UDP server program.`
- `package udpserver;`
- `import java.net.DatagramPacket;`
- `import java.net.DatagramSocket;`
- `public class UDPServer {`
- `public static void main(String[] args) {`
-
- `final int PORT = 9876;`
-
- `try (DatagramSocket serverSocket = new DatagramSocket(PORT)) {`
- `System.out.println("UDP Server is running on port " + PORT);`
-
- `byte[] receiveBuffer = new byte[1024];`
-
- `while (true) {`
- `DatagramPacket receivePacket = new DatagramPacket(receiveBuffer,`
- `receiveBuffer.length);`
-
- `// Receive packet`
- `serverSocket.receive(receivePacket);`
-
- `String clientMessage = new String(receivePacket.getData(), 0, receivePacket.getLength());`
- `System.out.println("Received from client: " + clientMessage);`

UDP Server Code

- `// Send response`
- `String response = "Hello from server!";`
- `byte[] sendBuffer = response.getBytes();`
- `DatagramPacket sendPacket = new DatagramPacket(`
- `sendBuffer, sendBuffer.length,`
- `receivePacket.getAddress(), receivePacket.getPort()`
- `);`
- `serverSocket.send(sendPacket);`
- `}`
- `} catch (Exception e) {`
- `e.printStackTrace();`
- `}`
- `}`
- `}`

UDP Client code

- `// UDPClient.java: A simple UDP client program.`
- `package udpclient;`
- `import java.net.DatagramPacket;`
- `import java.net.DatagramSocket;`
- `import java.net.InetAddress;`
- `public class UDPClient {`
- `public static void main(String[] args) {`
- `final String SERVER_IP = "localhost"; // or IP address`
- `final int SERVER_PORT = 9876;`
- `try (DatagramSocket clientSocket = new DatagramSocket()) {`
- `String message = "Hello from client!";`
- `byte[] sendBuffer = message.getBytes();`
- `InetAddress serverAddress = InetAddress.getByName(SERVER_IP);`
- `// Send packet to server`
- `DatagramPacket sendPacket = new DatagramPacket(sendBuffer,`
`sendBuffer.length, serverAddress, SERVER_PORT);`
- `clientSocket.send(sendPacket);`

UDP Client code ...

- `// Prepare to receive response`
- `byte[] receiveBuffer = new byte[1024];`
- `DatagramPacket receivePacket = new
DatagramPacket(receiveBuffer, receiveBuffer.length);`
- `clientSocket.receive(receivePacket);`
- `String serverResponse = new String(receivePacket.getData(),
0, receivePacket.getLength());`
- `System.out.println("Response from server: " +
serverResponse);`
- `} catch (Exception e) {`
- `e.printStackTrace();`
- `}`
- `}`
- `}`

Stream Socket vs. Datagram Socket

- Stream socket
 - A dedicated point-to-point channel between a client and server.
 - Use TCP (Transmission Control Protocol) for data transmission.
 - Lossless and reliable.
 - Sent and received in the same order.
- Datagram socket
 - No dedicated point-to-point channel between a client and server.
 - Use UDP (User Datagram Protocol) for data transmission.
 - May lose data and not 100% reliable.
 - Data may not be received in the same order as sent.

URL

- URL is an acronym for *Uniform Resource Locator* and is a reference (an address) to a resource on the Internet.
- URL has two main components:
 - Protocol identifier: For the URL `http://example.com`, the protocol identifier is `http`.
 - Resource name: For the URL `http://example.com`, the resource name is `example.com`.

URL ...

- The resource name is the complete address to the resource.
- **Host Name**
 - The name of the machine on which the resource lives.
- **File name**
 - The pathname to the file on the machine.
- **Port Number**
 - The port number to which to connect (typically optional).
- **Reference**
 - A reference to a named anchor within a resource that usually identifies a specific location within a file (typically optional).

Creating a URL

- `URL myURL = new URL("http://example.com/");`
- `URL page1URL = new URL(myURL, "page1.html");`
- `URL page2URL = new URL(myURL, "page2.html");`
- `URL page1BottomURL = new URL(page1URL, "#BOTTOM");`
- `new URL("http", "example.com",
"/pages/page1.html");`
- `URL url = new URL("http://example.com/hello%20world");`
 - `URI uri = new URI("http", "example.com", "/hello world/",
"");`
 - And then convert the URI to a URL.
 - `URL url = uri.toURL();`

URL Exception

- **MalformedURLException**
- try {
- URL myURL = new URL(...);
- }
- catch (MalformedURLException e) {
- *// exception handler code here*
- *// ...*
- }

Parsing a URL

- The URL class provides several methods that let you query URL objects.
- **getProtocol:-** Returns the protocol identifier component of the URL.
- **getAuthority:-** Returns the authority component of the URL.
- **getHost:-** Returns the host name component of the URL.
- **getPort:-** Returns the port number component of the URL. The getPort method returns an integer that is the port number. If the port is not set, getPort returns -1.
- **getPath:-** Returns the path component of this URL.
- **getQuery:-** Returns the query component of this URL.
- **getFile:-** Returns the filename component of the URL. The getFile method returns the same as getPath, plus the concatenation of the value of getQuery, if any.
- **getRef:-** Returns the reference component of the URL.

Example

- `import java.net.*;`
- `import java.io.*;`
-
- `public class ParseURL {`
- `public static void main(String[] args) throws Exception {`
-
- `URL aURL = new URL("http://example.com:80/docs/books/tutorial"`
- `+ "/index.html?name=networking#DOWNLOADING");`
- `System.out.println("protocol = " + aURL.getProtocol());`
- `System.out.println("authority = " + aURL.getAuthority());`
- `System.out.println("host = " + aURL.getHost());`
- `System.out.println("port = " + aURL.getPort());`
- `System.out.println("path = " + aURL.getPath());`
- `System.out.println("query = " + aURL.getQuery());`
- `System.out.println("filename = " + aURL.getFile());`
- `System.out.println("ref = " + aURL.getRef());`
- `}`
- `}`
- Here is the output displayed by the program:
 - `protocol = http`
 - `authority = example.com:80`
 - `host = example.com`
 - `port = 80`
 - `path = /docs/books/tutorial/index.html`
 - `query = name=networking`
 - `filename = /docs/books/tutorial/index.html?name=networking`
 - `ref = DOWNLOADING`

Reading Directly from a URL

- `import java.net.*;`
- `import java.io.*;`
- `public class URLReader {`
- `public static void main(String[] args) throws Exception {`
- `URL oracle = new URL("http://www.oracle.com/");`
- `BufferedReader in = new BufferedReader(`
- `new InputStreamReader(oracle.openStream()));`
- `String inputLine;`
- `while ((inputLine = in.readLine()) != null)`
- `System.out.println(inputLine);`
- `in.close();`
- `}`
- `}`

Connecting to a URL

- try {
- URL myURL = new URL("http://example.com/");
- URLConnection myURLConnection =
myURL.openConnection();
- myURLConnection.connect();
- }
- catch (MalformedURLException e) {
- // new URL() failed
- // ...
- }
- catch (IOException e) {
- // openConnection() failed
- // ...
- }

Reading from a **URLConnection**

- `import java.net.*;`
- `import java.io.*;`
-
- `public class URLConnectionReader {`
- `public static void main(String[] args) throws Exception {`
- `URL oracle = new URL("http://www.oracle.com/");`
- `URLConnection yc = oracle.openConnection();`
- `BufferedReader in = new BufferedReader(new`
`InputStreamReader(yc.getInputStream()));`
- `String inputLine;`
- `while ((inputLine = in.readLine()) != null)`
- `System.out.println(inputLine);`
- `in.close();`
- `}`

Writing to a URLConnection

```
• import java.io.*;
• import java.net.*;
•
• public class Reverse {
•     public static void main(String[] args) throws Exception {
•
•         if (args.length != 2) {
•             System.err.println("Usage: java Reverse "
•                 + "http://<location of your servlet/script>"
•                 + " string_to_reverse");
•             System.exit(1);
•         }
•
•         String stringToReverse = URLEncoder.encode(args[1], "UTF-8");
•
•         URL url = new URL(args[0]);
•         URLConnection connection = url.openConnection();
•         connection.setDoOutput(true);
•
•         OutputStreamWriter out = new OutputStreamWriter(
•             connection.getOutputStream());
•         out.write("string=" + stringToReverse);
•         out.close();
•
•         BufferedReader in = new BufferedReader(
•             new InputStreamReader(
•                 connection.getInputStream()));
•         String decodedString;
•         while ((decodedString = in.readLine()) != null) {
•             System.out.println(decodedString);
•         }
•         in.close();
•     }
• }
```

Network Interface

- the point of interconnection between a computer and a private or public network
- `java.net.NetworkInterface`
- `NetworkInterface` is useful for a multi-homed system, which is a system with multiple NICs.
- Using `NetworkInterface`, you can specify which NIC to use for a particular network activity.
- `NetworkInterface nif = NetworkInterface.getByName("bge0");`
- `Enumeration<InetAddress> nifAddresses = nif.getInetAddresses();`
- `Socket soc = new java.net.Socket();`
- `soc.bind(new InetSocketAddress(nifAddresses.nextElement(), 0));`
- `soc.connect(new InetSocketAddress(address, port));`

Retrieving Network Interfaces

- `import java.io.*;`
- `import java.net.*;`
- `import java.util.*;`
- `import static java.lang.System.out;`
- `public class ListNIFs`
- `{`
- `public static void main(String args[]) throws SocketException {`
- `Enumeration<NetworkInterface> nets = NetworkInterface.getNetworkInterfaces();`
-
- `for (NetworkInterface netIf : Collections.list(nets)) {`
- `out.printf("Display name: %s\n", netIf.getDisplayName());`
- `out.printf("Name: %s\n", netIf.getName());`
- `displaySubInterfaces(netIf);`
- `out.printf("\n");`
- `}`
- `}`
-
- `static void displaySubInterfaces(NetworkInterface netIf) throws SocketException {`
- `Enumeration<NetworkInterface> subIfs = netIf.getSubInterfaces();`
-
- `for (NetworkInterface subIf : Collections.list(subIfs)) {`
- `out.printf("\tSub Interface Display name: %s\n", subIf.getDisplayName());`
- `out.printf("\tSub Interface Name: %s\n", subIf.getName());`
- `}`
- `}`
- `}`

Listing Network Interface Addresses

- `public class ListNets {`
- `public static void main(String args[]) throws SocketException {`
- `Enumeration<NetworkInterface> nets =`
- `NetworkInterface.getNetworkInterfaces();`
- `for (NetworkInterface netint : Collections.list(nets))`
- `displayInterfaceInformation(netint);`
- `}`
- `static void displayInterfaceInformation(NetworkInterface netint) throws`
- `SocketException {`
- `out.printf("Display name: %s\n", netint.getDisplayName());`
- `out.printf("Name: %s\n", netint.getName());`
- `Enumeration<InetAddress> inetAddresses = netint.getInetAddresses();`
- `for (InetAddress inetAddress : Collections.list(inetAddresses)) {`
- `out.printf("InetAddress: %s\n", inetAddress);`
- `}`
- `out.printf("\n");`
- `}`
- `}`

- public class ListNetsEx {
- public static void main(String args[]) throws SocketException {
- Enumeration<NetworkInterface> nets = NetworkInterface.getNetworkInterfaces();
- for (NetworkInterface netint : Collections.list(nets))
- displayInterfaceInformation(netint);
- }
- static void displayInterfaceInformation(NetworkInterface netint) throws SocketException {
- out.printf("Display name: %s\n", netint.getDisplayName());
- out.printf("Name: %s\n", netint.getName());
- Enumeration<InetAddress> inetAddresses = netint.getInetAddresses();
- for (InetAddress inetAddress : Collections.list(inetAddresses)) {
- out.printf("InetAddress: %s\n", inetAddress);
- }
- out.printf("Up? %s\n", netint.isUp());
- out.printf("Loopback? %s\n", netint.isLoopback());
- out.printf("PointToPoint? %s\n", netint.isPointToPoint());
- out.printf("Supports multicast? %s\n", netint.supportsMulticast());
- out.printf("Virtual? %s\n", netint.isVirtual());
- out.printf("Hardware address: %s\n",
- Arrays.toString(netint.getHardwareAddress()));
- out.printf("MTU: %s\n", netint.getMTU());
- out.printf("\n");
- }
- }

- The following is sample output from the example program:
 - Display name: bge0
 - Name: bge0
 - InetAddress: /fe80:0:0:0:203:baff:fef2:e99d%2
 - InetAddress: /129.156.225.59
 - Up? true
 - Loopback? false
 - PointToPoint? false
 - Supports multicast? false
 - Virtual? false
 - Hardware address: [0, 3, 4, 5, 6, 7]
 - MTU: 1500
 -
 - Display name: lo0
 - Name: lo0
 - InetAddress: /0:0:0:0:0:0:0:1%1
 - InetAddress: /127.0.0.1
 - Up? true
 - Loopback? true
 - PointToPoint? false
 - Supports multicast? false
 - Virtual? false
 - Hardware address: null
 - MTU: 8232

Thanks